



UNIVERSITAT
POLITÈCNICA
DE VALÈNCIA



Escola Tècnica
Superior d'Enginyeria
Informàtica

Escuela Técnica Superior de Ingeniería Informática
Universidad Politécnica de Valencia

Predictor de Probabilidad de ValenBisi

PROYECTO I

Grado en Inteligencia Artificial

Autores: Vasyl Ishutin
Pablo Morant Belda
Gabriel Borràs Roger
Adrián Sabater Perales
Juan Álvaro López

Tutores: José Miguel Carot Sierra
Joaquín Martínez Minaya
Sandra Morales Martínez
Miguel Rebollo Pedruelo

Curso 2025-2026

1. INTRODUCCIÓN.....	2
1.1. Contexto y Motivación.....	2
1.2. Planteamiento del Problema: Predicción a Futuro y Probabilidad de Éxito.....	3
1.3. Objetivos del Proyecto.....	3
2. ESTADO DEL ARTE Y ALTERNATIVAS TECNOLÓGICAS.....	4
2.1. Modelos Clásicos vs. Redes Neuronales.....	4
2.2. Nuestra Elección: El Algoritmo Prophet.....	5
2.3. Por qué usamos el Prophet Clásico.....	6
3. METODOLOGÍA Y TECNOLOGÍAS.....	6
3.1. Metodología de Trabajo.....	6
3.2. Herramientas Utilizadas.....	7
4. INGENIERÍA DE DATOS Y ANÁLISIS EXPLORATORIO (EDA).....	7
4.1. El Problema de la Recolección: El "Agujero" de Datos.....	7
4.2. La Dinámica de la Ciudad: Estaciones Fantasma y Clausuradas.....	8
4.3. El Cruce de Datos con el Clima: El Reto de la Asincronía.....	9
4.4. Prevención de Fuga de Datos (Data Leakage).....	10
5. INGENIERÍA DE CARACTERÍSTICAS Y MODELADO PREDICTIVO.....	10
5.1. El Problema del Clima: ¿Por qué usamos Lógica Difusa?.....	10
5.2. Configuración del Modelo: Transformación Matemática "Logit" (Límites Físicos).....	11
5.3. La necesidad de aislar el entorno: Un Modelo por Estación.....	12
5.4. Aprendizaje de las Rutinas: Calendario, Estacionalidades y Fines de Semana.....	14
5.5. El Proceso de Entrenamiento (Fit).....	16
6. EVALUACIÓN Y DESPLIEGUE DEL MODELO.....	16
6.1. ¿Cómo sabemos si el modelo acierta? (Métricas de Error Base).....	16
6.2. Guardar el "Cerebro" del Modelo (El formato Pickle).....	17
6.3. Solución técnica: El "Parche Anti-Bug" de Prophet.....	18
7. EL MOTOR PROBABILÍSTICO: SIMULACIONES DE MONTE CARLO.....	18
7.1. El Método Monte Carlo (Creando Multiversos).....	18
7.2. El Rigor De Las Simulaciones.....	19
7.3. La Calculadora Predictiva (El Resultado Final).....	19
7.4. Despliegue Final: Desarrollo de la Aplicación Web.....	20
7.5. El Problema Con Los Límites: Padding.....	20
8. ANÁLISIS DEL IMPACTO METEOROLÓGICO Y SELECCIÓN DE PREDICTORES.....	21
8.1. El Experimento (Modelos A y B).....	21
8.2. El Descubrimiento: Filtrado de Ruido y Exclusión del Viento.....	21
9. CONSIDERACIONES ÉTICAS, SOCIALES Y ECONÓMICAS.....	22
9.1. Impacto Social y Medioambiental (ODS).....	22
9.2. Privacidad y Protección de Datos (RGPD).....	22
9.3. Viabilidad Económica.....	23
10. CONCLUSIONES Y TRABAJO FUTURO.....	23
10.1. Conclusiones del Proyecto.....	23
10.2. Trabajo Futuro.....	24
11. REFERENCIAS BIBLIOGRÁFICAS.....	24
12. GLOSARIO DE TÉRMINOS.....	25

1. INTRODUCCIÓN

1.1. Contexto y Motivación

Hoy en día, las ciudades intentan ser más sostenibles, reducir la huella de carbono y disminuir el uso del vehículo privado. En Valencia, el sistema público de bicicletas compartidas, conocido como "Valenbisi", se ha convertido en una herramienta clave para la movilidad urbana. Miles de ciudadanos lo utilizan a diario para ir a trabajar, a la universidad o para moverse por el centro de la ciudad. Sin embargo, a pesar de su popularidad, este sistema se enfrenta a un problema operativo muy común que genera gran frustración entre los usuarios: la falta de disponibilidad.

El uso de las bicicletas en una ciudad no es constante, sino que varía enormemente dependiendo de la hora del día, el día de la semana o incluso de la época del año. Por ejemplo, por las mañanas se producen desplazamientos masivos desde los barrios residenciales de la periferia hacia las zonas universitarias o los centros de negocios. Esta dinámica de movimiento provoca que las bicicletas se acumulen en unas zonas mientras otras se quedan completamente vacías. Esto genera dos escenarios críticos para el usuario:

- **El problema de las estaciones vacías:** El usuario sale de su casa con la intención de coger una bicicleta para ir a su destino, pero al llegar a la estación se encuentra con que no hay ninguna bicicleta disponible. Esto le obliga a caminar hacia otra estación lejana o, en el peor de los casos, a buscar un medio de transporte alternativo, perdiendo tiempo y confianza en el servicio.
- **El problema de las estaciones llenas:** El usuario consigue una bicicleta y realiza su trayecto con normalidad, pero al llegar a la estación destino descubre que todos los anclajes (bornas) están ocupados. Al no poder devolver la bicicleta, se ve forzado a deambular por la zona buscando otra estación con huecos libres, lo que supone un gran inconveniente si tiene prisa.

Tradicionalmente, las empresas que gestionan estos sistemas intentan solucionar este problema moviendo bicicletas de una estación a otra utilizando camiones de reparto. Sin embargo, esta solución es lenta, muy costosa económicamente y reacciona tarde al problema. Por ello, la motivación principal de este proyecto es utilizar la Inteligencia Artificial para anticiparnos al problema, dándole al usuario el poder de predecir cómo va a estar una estación en el futuro para que pueda planificar su viaje con seguridad.

1.2. Planteamiento del Problema: Predicción a Futuro y Probabilidad de Éxito

Normalmente, las aplicaciones actuales solo muestran una foto de lo que está pasando en ese mismo instante (ej. "Ahora mismo hay 4 bicis"). Pero, ¿qué ocurre si un usuario quiere planificar su semana?. Si hoy es lunes y un estudiante tiene un examen importante el miércoles a las 08:30 de la mañana, saber cuántas bicis hay hoy no le sirve de nada; necesita saber qué pasará en el futuro.

Por tanto, nuestro modelo no solo analiza el presente, sino que aprende patrones para predecir el futuro. Nuestro sistema es capaz de recibir una consulta para dentro de 3 días (o cualquier otra fecha futura) a una hora exacta, y proyectar cómo se comportará la estación en ese instante.

Además, los modelos predictivos tradicionales intentan adivinar un número exacto (por ejemplo, un modelo matemático básico podría predecir que el miércoles a las 08:30h habrá "2.4 bicicletas" en la estación). Para un usuario real, este dato resulta confuso y poco práctico. El usuario no quiere un número decimal con margen de error; el usuario solo quiere responder a una pregunta muy directa: *¿Voy a encontrar una bici cuando llegue, sí o no?*.

Por este motivo, en este trabajo hemos cambiado radicalmente el enfoque. En lugar de intentar adivinar el número exacto de bicicletas, nuestro modelo calcula la probabilidad de éxito. Es decir, gracias a nuestro sistema predictivo, el usuario podrá consultar el estado de su estación para dentro de varios días y obtener un mensaje útil y claro, como por ejemplo: "El próximo miércoles a las 08:30h tienes un 85% de probabilidad de encontrar al menos una bicicleta disponible en esta estación".

1.3. Objetivos del Proyecto

Para lograr construir este sistema, hemos marcado una serie de pasos fundamentales.

Objetivo General: Crear, entrenar y validar un modelo de Inteligencia Artificial capaz de predecir a futuro la probabilidad real de que haya bicicletas disponibles en las estaciones de Valenbisi, teniendo en cuenta tanto el calendario (día, hora, festivos, laborable o no, ...) como las condiciones climáticas.

Objetivos Específicos:

- **Auditoría y limpieza de datos:** Analizar y arreglar los fallos en los registros originales extraídos de Valenbisi (**Ayuntamiento de Valencia, s.f.**). Esto incluye solucionar grandes vacíos de información (meses donde no se guardaron datos) y limpiar el

sistema eliminando estaciones que estaban cerradas temporalmente o que eran simplemente pruebas de los técnicos.

- **Cruce de bases de datos:** Juntar los datos históricos de uso de las bicicletas con los datos meteorológicos pasados (temperatura, lluvia y viento). El reto aquí es alinear correctamente la hora exacta en la que se cogió la bicicleta con el clima que hacía en ese mismo minuto.
- **Entrenamiento para predicción a futuro:** Configurar el algoritmo Prophet para entrenar un cerebro independiente por cada estación física de la ciudad. El objetivo es que el modelo pueda proyectar esas rutinas hacia adelante en el tiempo (horizontes predictivos) respetando siempre los límites físicos reales (una estación nunca puede tener menos de 0 bicis ni más de las que caben en sus anclajes).
- **Transformación del clima mediante Lógica Difusa:** Las personas no cambiamos de comportamiento por un grado de diferencia (de 20°C a 21°C actuamos igual), pero sí cambiamos radicalmente si pasamos de 15°C a 5°C. Por ello, el objetivo es transformar los números exactos del clima en conceptos más humanos (por ejemplo, "clima ideal", "lluvia fuerte" o "viento molesto") para que el modelo lo entienda mejor.
- **Ingeniería de variables de calendario:** Inyectar variables temporales avanzadas en el algoritmo que le enseñen a diferenciar drásticamente entre la rutina estricta de movilidad de un día laborable y el comportamiento de un fin de semana o día festivo.
- **Simulaciones de riesgo (Monte Carlo):** Añadir al modelo matemático un sistema de simulaciones. Este sistema probará cientos de escenarios futuros posibles para calcular una probabilidad de éxito real y fácil de entender por cualquier ciudadano, en lugar de darle un número exacto y determinista.
- **Despliegue en Aplicación Web:** Construir una página web interactiva conectada por API a nuestros scripts, permitiendo a cualquier usuario introducir una estación y recibir al instante la predicción de bicicletas, el margen de error (intervalo de confianza) y la probabilidad matemática de éxito.

2. ESTADO DEL ARTE Y ALTERNATIVAS TECNOLÓGICAS

2.1. Modelos Clásicos vs. Redes Neuronales

A la hora de crear programas que intentan predecir datos en el futuro basándose en el pasado (lo que se conoce como análisis de series temporales), existen diferentes herramientas. Históricamente, en matemáticas y economía se han usado modelos clásicos como ARIMA.

Aunque los modelos ARIMA son muy conocidos, fallan estrepitosamente en nuestro caso por varios motivos. En primer lugar, se confunden cuando hay que predecir varios días a futuro, ya que el error se va acumulando y la línea de predicción pierde sentido. En segundo lugar, no saben manejar bien las rutinas mezcladas: a un ARIMA le cuesta mucho entender que a las 09:00h de un martes pasa una cosa, pero a las 09:00h de un domingo pasa algo totalmente distinto. Además, si en el historial de datos falta un mes entero de registros, el modelo clásico da un error informático y no puede continuar.

Para solucionar estos fallos, en los últimos años se han empezado a usar Redes Neuronales (especialmente los modelos llamados LSTM), que son sistemas de aprendizaje profundo mucho más potentes. El problema de las redes neuronales es que funcionan como una "caja negra". Es decir, son capaces de predecir muy bien, pero el algoritmo es tan complejo que no nos deja ver por qué ha tomado esa decisión. Para nuestro proyecto, era fundamental poder entender gráficamente cómo afecta un día festivo o una tormenta a las bicicletas, por lo que descartamos usar redes neuronales.

2.2. Nuestra Elección: El Algoritmo Prophet

Para encontrar el equilibrio perfecto entre acertar a futuro y poder entender qué está haciendo el programa, decidimos utilizar Prophet. Se trata de un algoritmo de código abierto desarrollado por el equipo de Meta (Facebook) (**Taylor & Letham, 2018**). Elegimos este modelo por varias razones clave que solucionaban todos nuestros problemas:

- **Predicción a largo plazo basada en rutinas:** Prophet no intenta adivinar el futuro fijándose solo en lo que pasó hace cinco minutos. En su lugar, descompone el tiempo en "piezas" o ciclos (la curva típica de un lunes, la curva de una mañana, la curva del verano). Esto es lo que nos permite pedirle una predicción para dentro de 3 días: el modelo simplemente coge las piezas de ese día futuro, las suma, y proyecta el resultado.
- **Resistencia a los datos faltantes:** Al contrario que los algoritmos antiguos, si a Prophet le faltan los datos de varios meses en nuestro registro histórico, no se rompe. Simplemente ignora ese hueco y sigue aprendiendo de los datos que sí tenemos, lo cual nos salvó la vida al procesar los archivos de Valenbisi.
- **Interpretabilidad total:** Nos permite generar gráficas donde vemos claramente el peso de cada decisión. Podemos ver exactamente cuántas bicicletas se dejan de usar en Valencia cuando es festivo nacional respecto a un día laborable normal.

2.3. Por qué usamos el Prophet Clásico

Durante la investigación tecnológica para este proyecto, valoramos usar versiones más modernas y experimentales de este algoritmo (como Prophetverse, una librería muy potente que se está popularizando en entornos de mucha capacidad de cálculo). Sin embargo, tras analizarlo, decidimos quedarnos con la versión original (Prophet clásico) por motivos muy prácticos y de sentido común:

- **Gestión de los límites físicos reales (Crecimiento Logístico):** El Prophet clásico tiene una función matemática extremadamente fiable llamada crecimiento logístico, sin embargo, hemos optado por realizar nuestra propia función logística para tener más control y mejor capacidad de visualización de los resultados. Esto es lo que nos permite ponerle "barreras" al futuro. Le decimos al programa: "Esta estación en concreto tiene un techo máximo de 20 anclajes y un suelo mínimo de 0 bicicletas". Así evitamos que al predecir a 3 días vista el modelo nos devuelva resultados absurdos o imposibles (como que habrá -4 bicicletas o 50 bicicletas en una estación donde sólo caben 15).
- **Rapidez y facilidad para exportar el modelo:** La versión clásica nos permitía exportar nuestro modelo una vez entrenado en un archivo de texto muy simple y ligero (formato JSON). Esto es fundamental para nuestro objetivo: significa que podemos tener una aplicación que cargue el "cerebro" del modelo en milisegundos y pueda responder a las preguntas del usuario sobre cuántas bicicletas habrá el próximo viernes de forma inmediata, sin necesidad de grandes servidores.

3. METODOLOGÍA Y TECNOLOGÍAS

3.1. Metodología de Trabajo

Para desarrollar este proyecto de forma organizada, nos hemos basado en el estándar de la industria conocido como CRISP-DM (Cross-Industry Standard Process for Data Mining), que es un marco de trabajo estándar y muy utilizado a la hora de estructurar proyectos de datos (Chapman et al., 2000). Esta metodología consiste en dividir el trabajo en seis fases lógicas que se van revisando a lo largo del proceso:

1. **Comprensión del problema:** Entender qué necesita el usuario de Valenbisi (saber si habrá bicis a futuro).
2. **Comprensión y recolección de los datos:** Descargar los datos de las estaciones y del clima, y analizarlos para ver si tienen errores y cómo se comportan.
3. **Preparación (Limpieza):** Borrar los datos que no sirven, rellenar huecos y juntar archivos.
4. **Modelado:** Configurar y entrenar el algoritmo Prophet para crear un "cerebro" predictivo independiente por cada estación.
5. **Evaluación:** Comprobar mediante métricas estadísticas si el modelo realmente acierta

en sus predicciones o si tiene un margen de error elevado.

6. **Despliegue:** Crear la aplicación web interactiva final conectada a nuestros scripts para que cualquier ciudadano pueda consultar las probabilidades de éxito desde su móvil u ordenador

3.2. Herramientas Utilizadas

Dado que nuestro objetivo era crear un sistema rápido y eficiente, hemos utilizado exclusivamente tecnologías de código abierto (gratuitas y accesibles):

- **Python 3:** El lenguaje de programación principal, estándar en el mundo de la Inteligencia Artificial.
- **Pandas:** Una librería de Python especializada en el manejo de grandes tablas de datos. Nos ha permitido abrir, limpiar y cruzar miles de archivos CSV de forma simultánea.
- **Prophet:** El algoritmo principal de predicción, que se encarga de realizar los cálculos matemáticos para proyectar el futuro.

4. INGENIERÍA DE DATOS Y ANÁLISIS EXPLORATORIO (EDA)

Antes de poder entrenar a un modelo de Inteligencia Artificial, es obligatorio asegurarse de que la información que se le va a dar es correcta. Si a un modelo se le introducen datos basura, las predicciones que devolverá serán basura (principio conocido en informática como *Garbage In, Garbage Out*). Por ello, esta fase de limpieza e ingeniería fue una de las más complejas del proyecto.

4.1. El Problema de la Recolección: El "Agujero" de Datos

Nuestro plan original era analizar un periodo exacto de dos años: desde el 1 de septiembre de 2023 hasta el 1 de septiembre de 2025. La idea era descargar el estado de cada estación cada 15 minutos durante ese tiempo.

Sin embargo, al empezar a analizar los archivos descargados (fase conocida como Análisis Exploratorio de Datos o EDA), descubrimos un fallo crítico. Durante la primavera de 2024, hubo un problema silencioso en el sistema de recolección original. Aunque los archivos se generaban todos los días, la información que contenían estaba "congelada". En la práctica, esto significaba que teníamos un vacío de información real de entre 4 y 5 meses.

Este agujero de datos era fatal para nuestro modelo. Prophet necesita ver cómo se comporta la ciudad en todas las estaciones del año para aprender la rutina (por ejemplo, en primavera la gente usa más la bici porque hace buen tiempo). Si le faltaban los datos de la primavera, el modelo iba a ser "ciego" a esa época del año.

La Solución (Ampliación Retrospectiva): Para solucionar esto sin inventarnos los datos, decidimos ampliar nuestro estudio hacia el pasado. Cambiamos la fecha de inicio al 1 de enero de 2023. Al hacer esto, le dimos al modelo casi un año extra de información histórica. De esta manera, aunque faltaran meses en 2024, el algoritmo ya había aprendido cómo se comporta Valencia en primavera gracias a los datos recopilados en 2023, salvando así el entrenamiento.

4.2. La Dinámica de la Ciudad: Estaciones Fantasma y Clausuradas

Otro gran descubrimiento de nuestro análisis fue entender que la red de Valenbisi no es un sistema estático; es una infraestructura viva que cambia con el tiempo. Al principio del proyecto en 2023, la ciudad contaba con 276 estaciones activas, pero observamos que los números dejan de encajar a medida que avanzaban los meses.

Para que nuestro modelo fuera preciso, tuvimos que programar filtros automáticos que limpiarán dos tipos de anomalías en la red física:

- **La estación de pruebas (ID 299):** Detectamos que entre junio y octubre de 2024 apareció de repente una estación con el identificador 299, llamada "Borna pruebas laboratorio madrid". Era evidente que no se trataba de una estación real en la calle, sino de un nodo técnico usado por los informáticos de la empresa para hacer pruebas. Si dejamos esta estación, el modelo intentaría predecir sus movimientos de forma absurda. Por tanto, la eliminamos por completo del proyecto.
- **Las estaciones destruidas o clausuradas:** El 25 de octubre de 2024, detectamos que la red pasó repentinamente de 276 a 273 estaciones. Al investigar, confirmamos que tres estaciones habían sido dadas de baja de forma definitiva (ID 105 Aularios UV, ID 168 Malvarrosa y ID 146 Av. Campanar).

El problema de las estaciones cerradas es muy grave para la Inteligencia Artificial. Si una estación se destruye, su número de bicicletas disponibles pasa a ser "0" para siempre. Si le

pasamos esto al modelo sin avisarle, el algoritmo pensará: "Ah, en esta calle nunca hay bicicletas a partir de octubre, debe ser una zona con mucha demanda". Estaría confundiendo una obra en la calle con el comportamiento humano. Para evitar esto, programamos un filtro que borra todo el historial de estas tres estaciones cerradas, asegurándonos de que el modelo solo aprende de estaciones reales y operativas.

4.3. El Cruce de Datos con el Clima: El Reto de la Asincronía

Una vez que teníamos los datos de las bicicletas limpios, el siguiente paso era cruzarlos con el historial del clima (temperatura, lluvia y viento). Sabemos que el clima afecta directamente a la decisión de usar una bicicleta: si llueve fuerte, el uso cae en picado.

El desafío técnico aquí es que los sensores de Valenbisi y los sensores de la API meteorológica (la base de datos del clima) no funcionan exactamente al mismo segundo (esto se llama asincronía).

Ejemplo del problema: Imaginemos que tenemos un registro que dice que un usuario cogió una bici a las 10:14h. Por otro lado, la base de datos del clima solo guarda registros a las horas en punto (a las 10:00h y a las 11:00h). Si intentamos hacer un cruce tradicional (buscar la misma hora exacta en los dos archivos), el programa dará error o dejará el clima en blanco, porque a las 10:14h no existe en el archivo del clima.

Nuestra Solución:

- **Interpolación de los datos:** En primer lugar, tuvimos que adaptar la frecuencia de la base de datos climatológica a la de los registros de bicicletas. Para esto, realizamos un remuestreo (resampling) para abrir huecos cada 15 minutos entre las mediciones horarias (por ejemplo, añadiendo las 10:15h, 10:30h y 10:45h entre las 10:00h y las 11:00h). A continuación, aplicamos una interpolación lineal de los datos para estimar matemáticamente los valores intermedios basándonos en los datos reales de la hora anterior y posterior. Con esto, obtuvimos una base de datos climática con intervalos de 15 minutos cuyas variables físicas (como la temperatura o el viento) varían de forma progresiva y natural.
- **Uso de la función `merge_asof`:** Una vez ya habíamos adaptado la base de datos del clima, tan sólo debíamos juntarla con el resto sin perder datos, para ello utilizamos una función muy avanzada de la librería Pandas (**McKinney, 2010**) llamada `merge_asof`. Esta función actúa de forma inteligente: cuando ve el registro de la bicicleta a las 10:14h, busca en el archivo del clima y, al no encontrar la hora exacta, "mira hacia atrás" y le asigna el clima más reciente que tenga sentido (en este caso, el clima de las

10:00h). Le pusimos un límite de tolerancia de 15 minutos máximo. Gracias a esto, logramos juntar millones de registros de bicicletas con su clima correspondiente sin generar errores informáticos.

4.4. Prevención de Fuga de Datos (Data Leakage)

El último gran obstáculo de la fase de procesamiento se descubrió al cruzar los archivos: las fechas no terminaban a la vez. Mientras que nuestros datos de Valenbisi llegaban, por ejemplo, hasta finales de septiembre, la base de datos oficial del clima había dejado de funcionar unas semanas antes.

En programación de datos, es común usar trucos para rellenar vacíos. Uno de ellos es el "arrastre hacia adelante" (*forward fill*), que consiste en copiar el último dato conocido para rellenar los días siguientes. Sin embargo, nos dimos cuenta de que si hacíamos esto, estaríamos cometiendo un error gravísimo conocido en Inteligencia Artificial como Fuga de Datos (*Data Leakage*).

Si el último dato del clima era del 1 de septiembre ("Soleado, 28°C"), la técnica de relleno copiaría "Soleado, 28°C" todos los días hasta finales de septiembre. Le estaríamos enseñando al modelo un clima artificial y congelado que nunca existió realmente, y el modelo aprendería patrones falsos.

Para proteger la integridad del proyecto, tomamos la decisión más estricta y profesional: aplicar un truncamiento. Recortamos todo nuestro proyecto exactamente en la fecha del último dato real del clima disponible. Preferimos sacrificar unas semanas de datos de bicicletas antes que entrenar a la Inteligencia Artificial con datos meteorológicos inventados.

5. INGENIERÍA DE CARACTERÍSTICAS Y MODELADO PREDICTIVO

Una vez que teníamos todos los datos limpios y cruzados en una única tabla gigante, el siguiente paso era prepararlos para que el modelo de Inteligencia Artificial los pudiera entender. A esta fase se le llama "Ingeniería de Características" (*Feature Engineering*), y consiste en transformar los datos brutos en variables inteligentes.

5.1. El Problema del Clima: ¿Por qué usamos Lógica Difusa?

Nuestra base de datos ya tenía la información meteorológica exacta de cada momento: grados Celsius para la temperatura, milímetros por hora para la lluvia y kilómetros por hora para el

viento. El primer instinto en ciencia de datos es pasarle estos números directamente al modelo, pero esto habría sido un gran error porque el comportamiento humano no es lineal.

Para entender esto, pongamos un ejemplo real. Si un usuario de Valenbisi sale de casa y hace 20°C, es muy probable que coja la bicicleta. Si al día siguiente hacen 21°C (un grado más), su comportamiento no va a cambiar en absoluto; seguirá cogiendo la bici. Sin embargo, si la temperatura baja de 15°C a 5°C, ese cambio drástico sí hará que el usuario decida coger el autobús o el metro en lugar de pedalear.

Con la lluvia pasa exactamente lo mismo. La diferencia entre que llueva 0 milímetros o 1 milímetro (cuatro gotas) apenas afecta al uso del servicio. Pero la diferencia entre 1 milímetro y 10 milímetros (una tormenta fuerte) vacía las calles de bicicletas de forma instantánea.

Si le dábamos a Prophet los números exactos, el algoritmo intentaría trazar una línea recta y asumiría que cada grado de temperatura afecta por igual, lo cual es falso.

Nuestra solución: Transformación por Lógica Difusa (Fuzzy Logic) Para enseñar a la Inteligencia Artificial a pensar como un ciudadano de Valencia, creamos un sistema de "Lógica Difusa". En lugar de usar los grados exactos, programamos una serie de funciones matemáticas que transforman el clima en "conceptos".

Creamos nuevas columnas en nuestros datos con nombres como `peso_temp_ideal`, `peso_frio_extremo` o `peso_lluvia_fuerte`. El programa evaluaba la temperatura real y le asignaba un porcentaje a estas columnas. Por ejemplo:

- Si hacían 22°C, la columna `peso_temp_ideal` valía un 100% (o 1.0) y la de `peso_frio_extremo` un 0%.
- Si hacían 12°C, el sistema no lo ponía todo a blanco o negro, sino que lo hacía "difuso": le daba un 50% al clima ideal y un 50% al clima frío.

Gracias a este truco, conseguimos que el modelo entendiera perfectamente que el clima funciona por "sensaciones" y umbrales, mejorando muchísimo su capacidad para predecir cuándo la gente iba a dejar las bicis aparcadas por culpa del mal tiempo.

5.2. Configuración del Modelo: Transformación Matemática "Logit" (Límites Físicos)

Si configuramos un modelo de Inteligencia artificial de forma estándar, este solo se fija en las tendencias matemáticas puras. Imaginemos una estación de Valenbisi situada en una universidad a las 08:00h; la tendencia de llegada de bicicletas es una línea que sube rápidamente. Si le pedimos al modelo predecir a las 09:00h, podría predecir "45 bicicletas". Sin embargo, si esa estación solo tiene 20 anclajes, es un resultado físicamente imposible. De igual forma, por la tarde, cuando la estación se vacía, podría predecir "-5 bicicletas", lo cual es igual

de absurdo.

Para respetar las leyes de la física, Prophet ofrece una configuración automática llamada "crecimiento logístico" (`growth='logistic'`). Sin embargo, decidimos no utilizarla y optar por un modelo de crecimiento lineal estándar (`growth='linear'`) combinándolo con una transformación matemática manual de los datos.

¿Por qué hacerlo manual? El ajuste por defecto de Prophet solo pone un límite físico a la tendencia general, pero los efectos "estacionales" (como los picos repentinos de la hora punta) se suman después. Esto provocaba que, en momentos de mucho movimiento, la suma de la tendencia y el pico horario volviera a romper los límites físicos de la estación, prediciendo capacidades imposibles.

Nuestra solución, mucho más robusta, consistió en aplicar una función matemática "Logit" antes de entrenar el modelo. Esta fórmula convierte la ocupación de la estación (0% a 100%) en un espacio matemático infinito. De este modo, Prophet trabaja libremente calculando tendencias lineales y estacionalidades sin chocar contra paredes. Una vez Prophet nos devuelve la predicción final (con todos los picos sumados), le aplicamos la fórmula inversa (la función Sigmoide). Esta actúa como un embudo que aplana suavemente cualquier número gigante, garantizando matemáticamente que la predicción final jamás supere el 100% de la capacidad de la estación, ni caiga por debajo del 0%.

5.3. La necesidad de aislar el entorno: Un Modelo por Estación

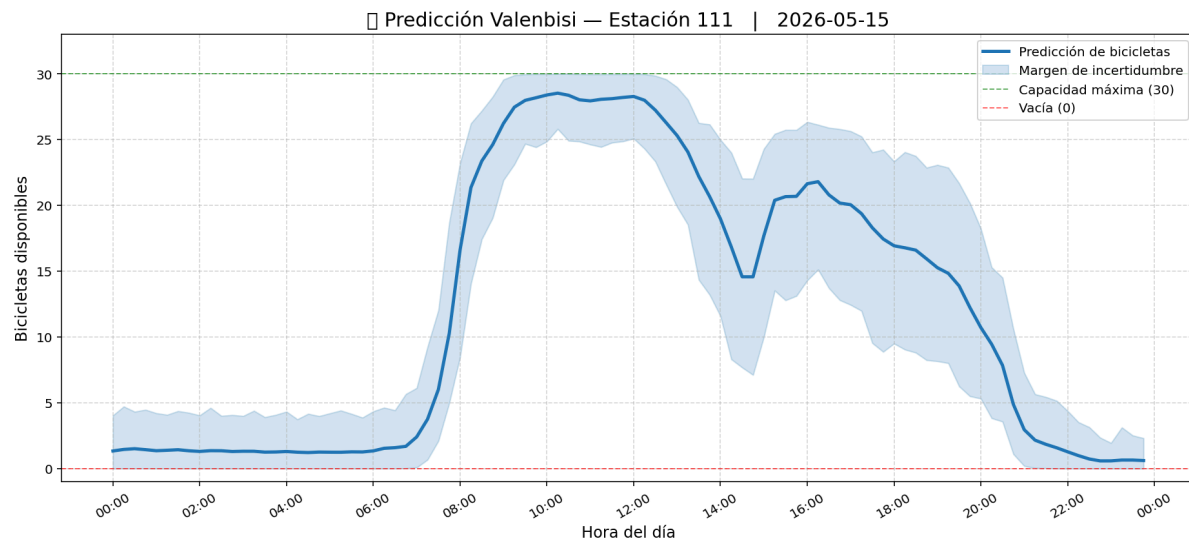
Durante la fase de diseño del algoritmo, nos dimos cuenta de que intentar crear un único "supermodelo" de Inteligencia Artificial para toda la ciudad era un error. El número de bicicletas no se puede generalizar, ya que depende de cada estación.

Por ejemplo, una estación situada en la zona universitaria se llena por completo a las 08:00h de la mañana los días laborables, ya que cientos de estudiantes llegan para ir a clase y saturan los anclajes. Por el contrario, una estación situada en un barrio residencial periférico se vacía a esa misma hora porque los ciudadanos cogen las bicicletas para iniciar su jornada. Si le diéramos todos los datos mezclados a Prophet, el algoritmo calcularía una "media" inexistente y fallaría en ambas estaciones.

La solución que implementamos fue aislar el entorno. Programamos un bucle en nuestro código que separa la información y entrena un modelo individual y exclusivo para cada una de las 273 estaciones operativas. Esto significa que nuestro sistema no es un solo cerebro, sino 273 minicerebros hiper-especializados, cada uno conociendo a la perfección las rutinas de su propia calle y sus propios límites físicos de capacidad.

Figura 1

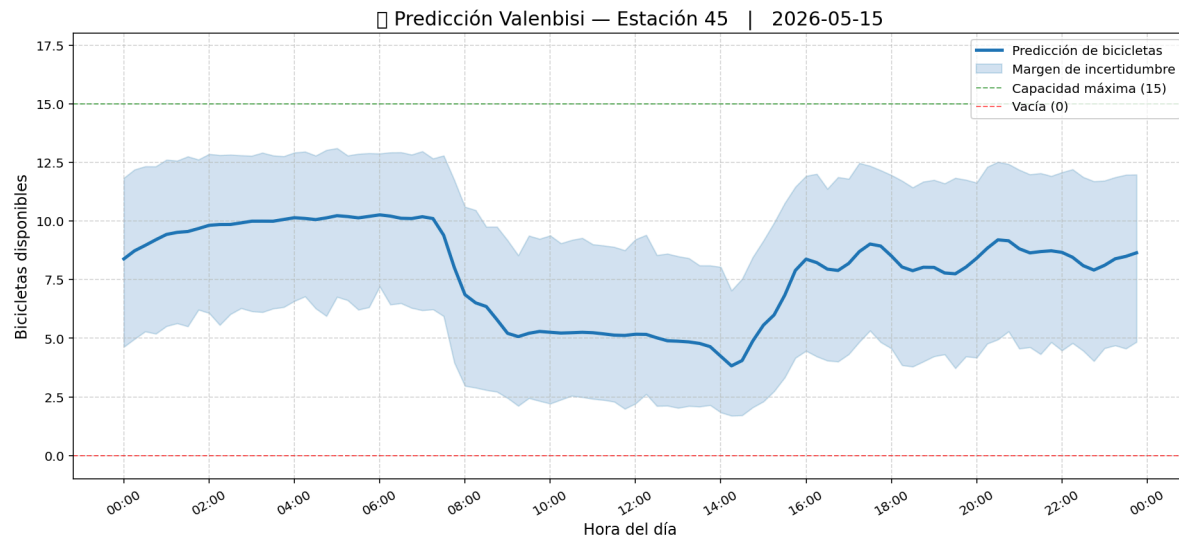
Estación perteneciente al recinto universitario



Nota. Gráfica de una estación universitaria entre semana con una tendencia muy marcada. Imagen extraída del script de graficación.

Figura 2

Estación ubicada fuera del recinto universitario



Nota. Gráfica de una estación aleatoria fuera de las universidades, predicción realizada sobre un día de la semana para extraer conclusiones fiables respecto a la gráfica anterior. Gráfica extraída del script de graficación.

5.4. Aprendizaje de las Rutinas: Calendario, Estacionalidades y Fines de Semana

El comportamiento de los usuarios de Valenbisi depende, entre otras cosas, del calendario. Para que los modelos pudieran predecir a futuro con precisión, tuvimos que activar sus "estacionalidades" (los ciclos que se repiten) y añadir variables específicas que le "explicaran" el calendario a la máquina.

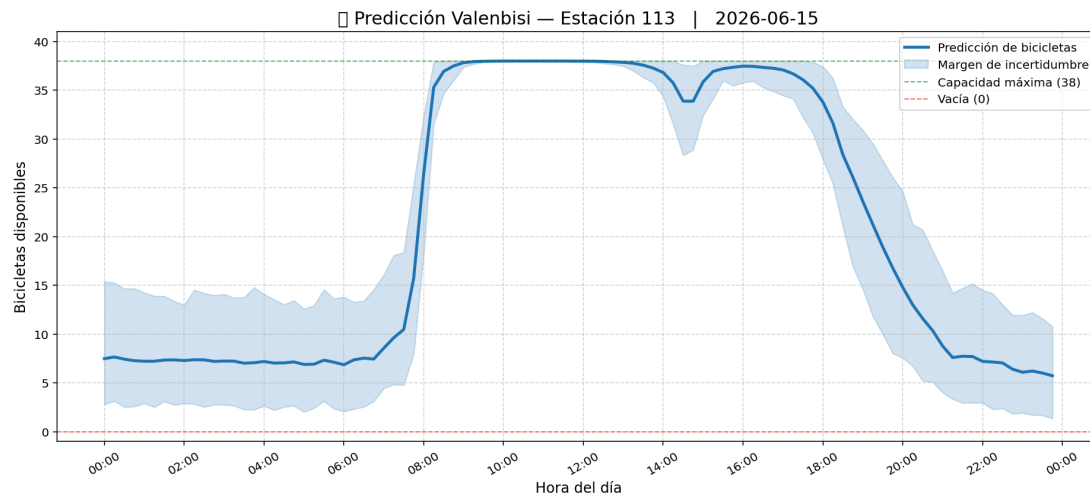
Le indicamos al modelo que prestara especial atención a varios factores temporales superpuestos:

- **El ciclo diario (*Daily Seasonality*):** El modelo aprendió que a las 03:00 de la madrugada el sistema está casi "congelado" porque nadie se mueve, mientras que a las 08:00h y a las 18:00h se producen picos extremos de movimiento por los horarios de trabajo y universidad.
- **El ciclo semanal (*Weekly Seasonality*):** El modelo entendió la diferencia radical entre un miércoles (diferencias enormes de bicis según la hora y la zona de oficinas) y un domingo (movimientos mucho más planos y orientados a zonas de ocio, playas o parques, independientemente de la hora).
- **El efecto de los Días Festivos (*Holidays*):** Este punto fue crítico. ¿Qué pasa si un martes cae en festivo nacional (por ejemplo, el 1 de mayo)? Si el modelo no lo sabe, aplicará la rutina normal de un martes laboral y fracasará estrepitosamente, porque la gente se comportará como si fuera domingo. Para solucionarlo, inyectamos en Prophet el calendario oficial de festivos de España (`country_name='ES'`).

La variable exógena "Laborable vs. Fin de Semana": Como mejora adicional para remarcar aún más esta diferencia, creamos una variable nueva. Si el día a predecir cae en sábado, domingo o festivo oficial, esta variable se enciende; si es un día de trabajo normal, se apaga. Al darle esta información extra tan clara, el modelo ajusta sus predicciones de forma mucho más precisa sin confundir los viajes de ocio con los viajes de trabajo. **Diferenciación entre días laborables y fines de semana:** Con el fin de perfeccionar la precisión del sistema, incorporamos una variable exógena que distingue entre jornadas de trabajo y días de descanso (sábados, domingos o festivos). Implementamos esta medida porque Prophet tiende a generar una curva única por estación que se ajusta al promedio de los datos; sin esta distinción, el modelo ofrecía un resultado intermedio que no representaba fielmente ni el comportamiento laboral ni el de ocio. Gracias a esta información adicional, el algoritmo logra separar nítidamente ambos patrones.

Figura 3

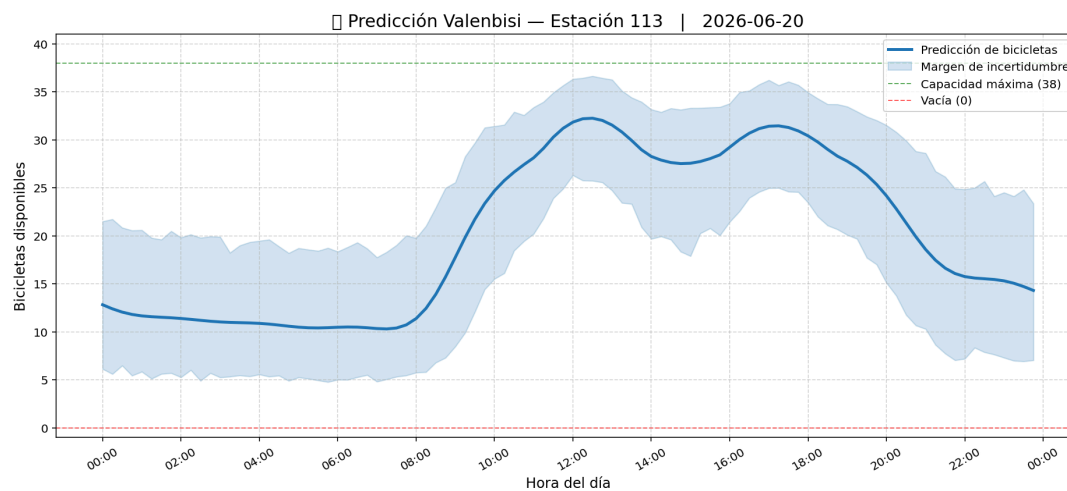
Estación ubicada dentro del recinto universitario entre semana



Nota. Gráfica de una estación universitaria con una tendencia muy marcada, predicción realizada sobre un día laborable para entender la diferencia de uso en laborables y festivos. Gráfica extraída del script de graficación.

Figura 4

Estación ubicada dentro del recinto universitario en fin de semana



Nota. Gráfica de una estación universitaria con una tendencia muy marcada, predicción realizada sobre un día festivo para entender la diferencia de uso en laborables y festivos. Gráfica extraída del script de graficación.

5.5. El Proceso de Entrenamiento (Fit)

Una vez establecidas todas las reglas (el crecimiento logístico, los límites de capacidad, las variables del clima difuso y los calendarios), procedimos a la fase de entrenamiento (`modelo.fit()`).

Durante esta fase, le pasamos al algoritmo todos los datos históricos que habíamos limpiado desde el 1 de enero de 2023. Lo que hace el ordenador internamente durante este proceso es pura estadística: analiza millones de filas de datos, calculando exactamente cuánto peso matemático tiene cada factor.

Por ejemplo, el modelo calcula cosas como: "Si es lunes a las 08:00h, suma 10 bicicletas. Pero si además está lloviendo fuerte (`peso_lluvia_fuerte = 1.0`), restan 6 bicicletas. Y como el techo es 15, asegúrate de no pasarte". Al terminar este proceso automático de cálculo, que tarda unos segundos por estación, obtenemos un modelo "entrenado", listo para recibir preguntas sobre el futuro.

6. EVALUACIÓN Y DESPLIEGUE DEL MODELO

Una vez que el modelo ha sido entrenado y ha aprendido las rutinas de la ciudad de Valencia, el siguiente paso lógico en cualquier proyecto de Ciencia de Datos es comprobar si realmente es bueno haciendo su trabajo y, posteriormente, prepararlo para que un usuario real pueda utilizarlo en una aplicación.

6.1. ¿Cómo sabemos si el modelo acierta? (Métricas de Error Base)

Antes de calcular probabilidades o tomar decisiones de negocio, necesitábamos asegurarnos de que la línea central de predicción (la estimación matemática pura de Prophet) era coherente con la realidad. Para ello, utilizamos un riguroso sistema de evaluación algorítmica llamado "Validación Cruzada" (Cross-Validation).

En lugar de conformarnos con medir el acierto del modelo una sola vez, este método divide todo el historial temporal de datos en diferentes bloques. El sistema oculta deliberadamente al modelo el último tramo de información (por ejemplo, el último mes de registros de las bicicletas) y le obliga a predecir qué ocurrió basándose únicamente en los meses anteriores. A continuación, comparamos la predicción matemática que ha generado el algoritmo con lo que realmente ocurrió ese mes. Al realizar este proceso de "ocultar y predecir" múltiples veces sobre diferentes ventanas temporales, nos aseguramos de que el modelo no se limite a "memorizar" el pasado, sino que realmente haya comprendido la lógica y los patrones de

comportamiento de los ciudadanos.

Tras ejecutar esta validación intensiva en toda la red de Valenbisi, consolidamos una tabla general de rendimiento. Para interpretar estos resultados, extrajimos tres métricas matemáticas estándar:

MAE (Error Absoluto Medio): Nos dice, en promedio, por cuántas bicicletas se desvía nuestra predicción de la realidad. A nivel global, nuestro MAE promedio en toda la ciudad es de 4.45 bicicletas. Es decir, de forma general, el modelo suele equivocarse por algo más de 4 bicicletas arriba o abajo en cada estación.

RMSE (Raíz del Error Cuadrático Medio): Esta métrica es más estricta que el MAE, ya que la fórmula matemática "castiga" severamente los errores muy graves. Si el modelo suele fallar por 2 bicis, pero un día falla por 15 debido a un evento anómalo (como una mascletá o una lluvia torrencial imprevista), el RMSE sube desproporcionadamente para alertarnos. Nuestro RMSE medio global es de 5.78 bicicletas. Que este valor se mantenga tan cercano al MAE es una excelente noticia: significa que el modelo es muy estable y rara vez produce predicciones disparatadas o "picos de error" incontrolables.

Score (Puntuación de Precisión Relativa): Se trata de una métrica que resume la fiabilidad del modelo en una nota global (del 0 al 1), ponderando los errores cometidos en función de la escala matemática de cada estación. Teniendo en cuenta que la capacidad media de una estación de Valenbisi es de 19.8 anclajes, hemos obtenido un espectacular Score medio global de 0.775 (77.5% de precisión en captar la varianza).

El patrón de predictibilidad de la red: Al no centrarnos en qué estación individual es la "mejor" o la "peor", sino en la estadística general, el Score nos revela un patrón muy claro sobre cómo respira la ciudad. Las zonas que presentan las puntuaciones más bajas (con un Score alrededor de 0.66 - 0.70) corresponden a estaciones con comportamientos muy erráticos o con poca capacidad (en torno a 15 anclajes). En estas estaciones pequeñas, un hecho puntual e impredecible, como que un grupo de cuatro amigos retire bicicletas al mismo tiempo, altera drásticamente los porcentajes.

Por el contrario, en las grandes arterias de la ciudad y estaciones de alta capacidad (30 a 40 anclajes), donde existe una rutina de movilidad marcada por horarios de oficina, universidades y transporte público, el modelo despliega su máxima potencia matemática. Aquí comprende a la perfección los picos y valles de uso, alcanzando puntuaciones de excelencia estadística con un Score que supera el 0.90 (90%).

Equivocarse por un margen promedio de ~4 bicicletas en estaciones donde operan cientos de personas al día demuestra que nuestra IA ha interiorizado correctamente el pulso de la ciudad, dándonos luz verde para pasar a la siguiente fase: la simulación probabilística final.

6.2. Guardar el "Cerebro" del Modelo (El formato Pickle)

El proceso de entrenar un modelo analizando millones de datos del pasado puede tardar varios minutos. Evidentemente, un usuario en la calle que consulta su móvil no puede esperar 10 minutos a que el modelo se entrene desde cero para saber si hay bicicletas.

Por tanto, una vez que el modelo está entrenado, hay que "guardar su cerebro" en un archivo. En el mundo de la Inteligencia Artificial, esto se conoce como serialización. Para esto, optamos por los archivos formato pickle.

6.3. Solución técnica: El "Parche Anti-Bug" de Prophet

Durante esta fase de guardado, nos enfrentamos a un problema informático muy conocido en la comunidad de desarrolladores de Prophet. Cuando guardas un modelo estándar en formato JSON y lo vuelves a cargar, funciona perfectamente. Sin embargo, cuando usas el crecimiento logístico (los límites de capacidad máxima y mínima que le pusimos a las estaciones), la herramienta de guardado tiene un fallo interno o bug.

Al intentar cargar nuestro modelo para hacer una predicción, el programa daba error porque Prophet "olvidaba" las escalas matemáticas del suelo y el techo (`y_scale` y `t_scale`).

En lugar de rendirnos o cambiar de formato, programamos un "parche" defensivo en nuestro código. Nuestro script ahora lee el archivo Pickle, detecta si el modelo ha perdido la memoria de las escalas y, de forma automática, restaura manualmente los valores originales antes de hacer la predicción. Gracias a esta solución técnica, logramos tener un sistema de guardado y carga robusto, instantáneo y listo para el mundo real.

7. EL MOTOR PROBABILÍSTICO: SIMULACIONES DE MONTE CARLO

Esta es la sección central del proyecto y la que aporta un valor real al ciudadano. Como hemos comentado desde la introducción, decirle a un usuario que mañana habrá "2.4 bicicletas" no le soluciona la vida. Necesitamos darle una métrica de riesgo real, es decir, un porcentaje de éxito. Para lograr esto, tuvimos que acoplar a nuestro modelo matemático un motor de simulaciones.

7.1. El Método Monte Carlo (Creando Multiversos)

Para calcular la probabilidad real de que ocurra algo, en estadística se utiliza una técnica llamada Simulaciones de Monte Carlo. Su funcionamiento es muy intuitivo: en lugar de predecir un único futuro exacto, el ordenador utiliza la incertidumbre y el error histórico del modelo para simular cientos de futuros posibles (como si fueran universos paralelos).

En nuestro código, utilizamos la función `predictive_samples` de Prophet. Cuando el usuario

pregunta: "¿Habrá bicis mañana a las 09:00h?", nuestro programa no tira una sola línea de predicción. Lo que hace es generar rápidamente cientos de líneas distintas que representan lo que podría pasar basándose en cómo se ha comportado el clima y la gente en el pasado. Unas simulaciones serán muy optimistas (marcarán 10 bicis) y otras serán pesimistas (marcarán 0 bicis).

7.2. El Rigor De Las Simulaciones

En la ciencia de datos, cuantas más simulaciones hagas, más exacto y estable es el resultado matemático. Al principio, nos planteamos generar un número bajo de simulaciones por consulta (por ejemplo, 150 futuros posibles) con el objetivo de priorizar la velocidad informática y conseguir que el sistema respondiera en una fracción de segundo (latencia casi cero).

Sin embargo, aquí nos encontramos con un problema grave: la "variabilidad estadística" (o ruido matemático). Si dejábamos el número de simulaciones demasiado bajo, el resultado se volvía inestable. Es como lanzar una moneda al aire solo diez veces: a veces obtendrás un 70% de caras y otras un 30% por puro azar. Si aplicamos esto a la aplicación, un usuario podía consultar la estación a las 10:00h y recibir una probabilidad de éxito del 60%; pero si por casualidad refrescaba la pantalla un segundo después, el modelo, al calcular tan pocos universos, le daba un resultado diferente, digamos 85%. Esto destruiría por completo la confianza del usuario en el sistema.

Por ello, tomamos una decisión de diseño radical: priorizar el rigor matemático absoluto. Escalamos el sistema para que ejecute cerca de 10.000 simulaciones (futuros posibles) cada vez que un usuario hace una consulta. Es la diferencia entre hacer una encuesta rápida en la calle a tres personas y realizar un censo masivo: al explorar 10.000 universos paralelos, eliminamos todo el "ruido" estadístico y forzamos al modelo a converger hacia la probabilidad real. De esta manera, garantizamos una seguridad innegociable: la certeza de que cada vez que el usuario pregunte, reciba una respuesta estadísticamente robusta, inamovible y 100% responsable.

7.3. La Calculadora Predictiva (El Resultado Final)

La lógica matemática que programamos es muy sencilla pero extremadamente efectiva. El programa hace lo siguiente:

1. Revisa las 10000 simulaciones para la hora que ha pedido el usuario.
2. Cuenta en cuántas de esas simulaciones la estación tiene 1 bicicleta o más (es decir, en cuántos "futuros" el usuario tiene éxito).
3. Si en 7000 simulaciones hay bicicletas y en 3000 no las hay, el programa hace una simple regla de tres (7000/10000).
4. El resultado final se le muestra al usuario: "Probabilidad de encontrar bicicleta: 70%".

Con esta arquitectura, hemos logrado transformar un algoritmo matemático muy denso en una calculadora de riesgo amigable, interactiva y pensada exclusivamente para mejorar la

experiencia de los ciudadanos de Valencia.

7.4. Despliegue Final: Desarrollo de la Aplicación Web

El objetivo final de cualquier proyecto de Ciencia de Datos no es dejar el código en el ordenador del programador, sino llevarlo a las manos del usuario final. Para lograrlo, dimos un paso más allá de la simple consola de comandos y desarrollamos una aplicación web interactiva.

Para la construcción de esta interfaz, nos apoyamos en herramientas de última generación. Utilizamos Google AI Studio para agilizar la estructuración y generación de ciertos bloques de código de la interfaz, y empleamos librerías específicas de Python como Antigravity para gestionar el entorno web y hacer que la programación fuera más fluida y ágil.

Arquitectura de la conexión (APIs y Scripts): La arquitectura de nuestra página web funciona mediante conexiones invisibles llamadas APIs. El flujo que hemos programado es el siguiente:

1. El usuario entra en la página web desde su móvil u ordenador e introduce el número de la estación a la que quiere ir y la fecha futura.
2. La página web captura esa petición y llama a nuestros scripts de Python en segundo plano.
3. El script busca instantáneamente el archivo `.pkl` (el cerebro entrenado) exclusivo de esa estación en concreto y lo carga en memoria.
4. Se ejecutan las simulaciones estocásticas de Monte Carlo y se devuelven los datos a la pantalla del usuario.

En lugar de mostrar números confusos, nuestra web traduce todo este complejo proceso matemático en tres resultados claros y directos:

- **Predicción central:** El número estimado de bicicletas que habrá en ese momento exacto.
- **Intervalo de confianza:** Un margen de error matemático (ej. "Entre 2 y 6 bicicletas") para que el usuario entienda la volatilidad de esa hora.
- **Probabilidad real:** El porcentaje definitivo (ej. "85% de posibilidades de éxito") calculado mediante nuestras simulaciones.

7.5. El Problema Con Los Límites: Padding

Nuestro modelo como tal da un resultado lineal, que nos da un valor lineal.

Decidimos hacer manualmente una función logit de enlace para tener más control sobre el resultado final. Pero la función de enlace tiene un problema. Para que el resultado sean valores extremos, como 0 o el máximo de la estación, nuestro modelo lineal tiene que dar un valor a la función de enlace cercana al infinito o al menos infinito. Como esto es imposible, a nuestro modelo le costaba mucho acercarse a valores extremos. Esto es un problema porque nos

interesa mucho que sea preciso en valores cercanos a 0 o al máximo, por lo que decidimos aplicar un efecto imán a los extremos de cada estación.

A valores predecidos cercanos a 0, se aplica una penalización cada vez más grande. De este modo si nuestro modelo predice 5/10 bicis no sufrirá ningún cambio, pero si predice 1/10 bicis el resultado bajará de forma proporcional a su cercanía con 0.

en nuestro modelo por ejemplo 1/10 bicis = 0/10 bicis. Esto hace que sea un modelo más atrevido en los extremos, y en nuestras pruebas, lo hacía un modelo más preciso.

Además, abordamos el problema de la varianza. El modelo original asumía homocedasticidad (varianza constante en todo momento). En estaciones con alta incertidumbre, esto provocaba que las simulaciones de Montecarlo arrojaran escenarios imposibles (simulaciones con 10 bicis cuando la media real tendía a 0). Esto inflaba los porcentajes de forma irreal

entonces añadimos que la varianza bajara de forma proporcional si se acerca a los extremos (igual que la media), y esto mejoraba el modelo.

Por último, para el cálculo de la probabilidad final, redefinimos nuestro concepto de "éxito". En lugar de considerar como válida cualquier simulación que arrojará al menos 1 bicicleta, establecimos un umbral de seguridad del 12% de la capacidad máxima de la estación. Si una simulación de Montecarlo no supera este umbral, se considera un "fracaso"

8. ANÁLISIS DEL IMPACTO METEOROLÓGICO Y SELECCIÓN DE PREDICTORES

Durante el desarrollo del proyecto, partimos de la hipótesis de que la incorporación de variables meteorológicas (lluvia, temperatura y viento) optimizaría la precisión del modelo, asumiendo que el estado del tiempo condiciona el uso de la bicicleta. Sin embargo, tras someter los predictores a un análisis de impacto, descubrimos que no todos afectaban al modelo de la misma manera.

8.1. El Experimento (Modelos A y B)

Para evaluar matemáticamente el impacto real del clima y refinar la selección de características (*feature selection*), diseñamos un Test A/B para contrastar el rendimiento de dos configuraciones del algoritmo Prophet:

- **Modelo A (Línea base):** Entrenado exclusivamente con variables temporales y de calendario (hora, día de la semana y festivos).
- **Modelo B (Meteorológico extendido):** Incluía la base del Modelo A más las variables transformadas mediante lógica difusa: temperatura, lluvia y velocidad del viento.

8.2. El Descubrimiento: Filtrado de Ruido y Exclusión del Viento

Al comparar las métricas y las gráficas de probabilidad a 24 horas, observamos que el Modelo B mostraba un comportamiento dual. Mientras que la lluvia y la temperatura aportan valor en escenarios extremos, la variable del viento generaba una distorsión severa en las predicciones. Al introducir el viento, el intervalo de confianza de Prophet (la banda de incertidumbre que rodea la predicción) se ensanchaba notablemente. Esto se debe a que el uso de Valenbisi está fuertemente Target-driven (guiado por la rutina estricta de estudios o trabajo). Los usuarios habituales mantienen su patrón de desplazamiento matutino o vespertino independientemente de las ráfagas de viento, lo que convertía a esta variable en un factor puramente aleatorio para el modelo.

Conclusión del análisis: Al obligar al algoritmo a correlacionar el viento con la demanda, este intentaba patronar fluctuaciones inexistentes, introduciendo un "ruido estadística" que penaliza la precisión global. Por ello, tras analizar el efecto individual de los predictores, se decidió retirar definitivamente la variable viento del modelo final, manteniendo únicamente la temperatura y la lluvia como los únicos estimadores meteorológicos válidos junto al calendario.

9. CONSIDERACIONES ÉTICAS, SOCIALES Y ECONÓMICAS

En el desarrollo de cualquier tecnología que vaya a ser usada por ciudadanos reales, no solo hay que tener en cuenta que el código funcione, sino también el impacto que tendrá en la sociedad.

9.1. Impacto Social y Medioambiental (ODS)

Este proyecto está directamente alineado con los Objetivos de Desarrollo Sostenible (ODS) de las Naciones Unidas, concretamente con el ODS 11: Ciudades y Comunidades Sostenibles.

Uno de los mayores problemas para convencer a la gente de que deje el coche en casa y use la bicicleta pública es la incertidumbre. Si una persona llega tarde al trabajo una vez porque su estación estaba vacía, es probable que al día siguiente vuelva a coger su coche privado. Al darle al usuario una herramienta en su móvil que le asegure al 85% que tendrá una bicicleta disponible, eliminamos esa barrera de desconfianza. Fomentar el uso eficiente de Valenbisi ayuda a descongestionar el tráfico de Valencia, reduce el ruido ambiental y disminuye drásticamente las emisiones de CO2.

9.2. Privacidad y Protección de Datos (RGPD)

Hoy en día, las aplicaciones de movilidad suelen rastrear los datos de los usuarios (dónde

viven, a dónde van, a qué hora salen de casa). En nuestro proyecto, hemos garantizado la privacidad desde el diseño, cumpliendo estrictamente con el Reglamento General de Protección de Datos (RGPD).

Nuestro modelo no utiliza datos personales en ningún momento. La Inteligencia Artificial no sabe, ni le importa, quién coge la bicicleta. El modelo solo cuenta "unidades físicas" (sumar 1 bicicleta, restar 1 bicicleta) en una estación concreta. Al no haber GPS individualizado ni perfiles de usuario, es imposible rastrear a una persona real, haciendo que esta tecnología sea 100% segura para la privacidad de los valencianos.

9.3. Viabilidad Económica

Desde el punto de vista del negocio y la administración pública, este proyecto es extremadamente barato de mantener. Al haber utilizado exclusivamente lenguajes y librerías de código abierto (Open Source como Python y Prophet), el Ayuntamiento de Valencia no tendría que pagar ninguna licencia de software millonaria a empresas privadas.

10. CONCLUSIONES Y TRABAJO FUTURO

10.1. Conclusiones del Proyecto

El objetivo principal de este proyecto era crear un sistema capaz de predecir la disponibilidad futura de Valenbisi. Tras meses de desarrollo, podemos afirmar que el objetivo se ha cumplido con éxito, pero el camino nos ha enseñado que la Inteligencia Artificial en el mundo real es muy distinta a la teoría que se enseña en las aulas.

En primer lugar, hemos comprobado que el trabajo más duro no es programar la IA, sino limpiar y entender los datos. Lidar con meses de registros perdidos por fallos en los scripts de recolección de la fuente original, detectar estaciones "fantasma" de prueba o programar cruces de datos asíncronos (`merge_asof`) fue el verdadero reto del proyecto.

En segundo lugar, hemos logrado cambiar el enfoque clásico de la predicción. Entendimos que predecir un número decimal (ej. "2.4 bicicletas") no tiene valor para el ciudadano. Al acoplar el método de Monte Carlo y realizar simulaciones, logramos transformar un cálculo matemático abstracto en una probabilidad real de éxito ("Tienes un 80% de posibilidades de encontrar bici"). Este cambio de enfoque convierte nuestro código en un producto útil y directo para la sociedad.

En tercer lugar, nuestro trabajo gira en torno al usuario. Nos hemos centrado en analizar las variables que podrían ser significativas tanto matemáticamente como psicológicamente. Nos interesamos en cómo el usuario percibe el clima, ya que nuestra principal prioridad es la comodidad del servicio de ValenBisi, y nuestras predicciones lo toman en cuenta. Por esto

mismo, con estrategias como la aplicación del 'padding', dejamos claro que preferimos sacrificar las predicciones reales del modelo a cambio de dar un resultado que pueda asegurar la toma de decisión correcta del usuario.

10.2. Trabajo Futuro

A pesar del éxito del modelo, la tecnología avanza muy rápido y existen áreas de mejora que proponemos como futuras líneas de investigación:

- **Conexión entre estaciones (Redes de Grafos):** Actualmente, nuestro modelo mira cada estación de forma aislada. Si la estación A se vacía, nuestro modelo no avisa a la estación B (que está en la calle de al lado). En el futuro, se podría utilizar una tecnología llamada Redes Neuronales de Grafos, que permitiría al modelo entender que las estaciones están conectadas como una telaraña, mejorando las predicciones de todo el barrio a la vez.
- **Modelado Adaptativo por Estación:** Actualmente aplicamos un único algoritmo (Prophet) a toda la red. Sin embargo, nuestra validación demostró que las estaciones pequeñas y erráticas obtienen puntuaciones de Score menores que las grandes arterias rutinarias. Un avance clave para el futuro sería desarrollar una "arquitectura multi-modelo" que utilice estas métricas de error para decidir automáticamente qué algoritmo emplear en cada zona. Así, mantendríamos Prophet para las estaciones predecibles, pero cambiaríamos dinámicamente a algoritmos más reactivos a corto plazo (como Random Forest o Redes LSTM) para las estaciones más caóticas, construyendo un verdadero "traje a medida" para las necesidades de cada estación individual.
- **Conexión con la API oficial de ValenBisi:** Actualmente, hemos trabajado con volcados de datos históricos estáticos para construir y validar el modelo predictivo. El salto lógico para llevar valenBA a un entorno de producción real sería integrarlo de forma directa y automatizada con la API pública oficial (la fuente de datos en vivo del Ayuntamiento y JCDecaux). Esto no solo permitiría que el modelo se reentrenara y se hiciera más inteligente día a día sin intervención humana, sino que, de cara a la interfaz, el usuario podría ver en una misma pantalla cuántas bicicletas hay ahora mismo y nuestra predicción de cuántas habrá dentro de una hora, fusionando el estado presente y el probabilístico futuro en una sola herramienta integral de movilidad.

11. REFERENCIAS BIBLIOGRÁFICAS

- Ayuntamiento de Valencia. (s.f.). *Portal de Datos Abiertos del Ayuntamiento de Valencia: Repositorio histórico del uso y estado del servicio público Valenbisi*. <https://opendata.vlci.valencia.es>
- Chapman, P., Clinton, J., Kerber, R., Khabaza, T., Reinartz, T., Shearer, C., & Wirth, R. (2000). *CRISP-DM 1.0: Step-by-step data mining guide*. SPSS.
- McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Austin, TX.
- Taylor, S. J., & Letham, B. (2018). Forecasting at scale. *The American Statistician*, 72(1),

12. GLOSARIO DE TÉRMINOS

- **CSV:** Archivo de texto plano donde los datos están separados por comas, muy utilizado para guardar tablas de datos.
- **EDA (Análisis Exploratorio de Datos):** Fase en la que los programadores investigan los datos originales para buscar errores, agujeros de información o anomalías antes de empezar a programar.
- **Pickle:** Formato de archivo de texto muy ligero que se usa para guardar configuraciones o, en nuestro caso, el "cerebro" del modelo entrenado.
- **Lógica Difusa:** Técnica de programación que clasifica los números en conceptos (ej. pasar de 15°C a "Clima frío") para que las máquinas entiendan mejor los contextos humanos.
- **Monte Carlo:** Método estadístico que consiste en simular el mismo evento cientos de veces con pequeñas variaciones para calcular la probabilidad real de que algo ocurra.